TOKEN DISPENSER APPLICATION

AIM

To develop an Android application to implement a Token Dispenser system that generates unique tokens for users in a queue-like system using Android Studio Integrated Development Environment.

APPARATUS REQUIRED

- Computer
- Android Studio software

THEORY

Introduction to Android

Android is an open-source mobile operating system based on a modified version of the Linux kernel, designed primarily for touchscreen mobile devices such as smartphones and tablets. Android applications are written predominantly in Java or Kotlin programming languages and are compiled into bytecode for the Android Runtime (ART), which executes the applications on Android devices.

Token Dispenser System Overview

The Token Dispenser App generates unique tokens for users in a queue-like system. The application allows users to input their name and receive a token number upon clicking a button. The token is displayed, and a list of previously generated tokens is maintained and shown in a ListView. This experiment demonstrates how to manage dynamic lists and use array adapters in Android for displaying data. It also introduces handling user input, generating unique tokens, and managing collections like arrays and lists.

User Interface Components

- **EditText:** Text input field for capturing the user's name
- **Button:** Interactive element to trigger token generation
- **TextView:** Display component for showing the current issued token
- **ListView:** List display component for showing all previously generated tokens

Event Handling and Data Flow

- User enters their name in the EditText field
- User clicks the 'Get Token' button
- Token count is incremented
- Token information is created and displayed in TextView
- Token is added to the ArrayList and ListView is updated using ArrayAdapter

PROCEDURE

- Open Android Studio and click on File → New → New project
- Type the application name as 'TokenDispenser' and click Next
- Select the Minimum SDK as required and click Next
- Select the Empty Activity and click Next
- Click Finish
- Click on app → java → com.example.tokendispenser → MainActivity.java
- Type the Java program code for the main activity functionality
- Click on app → res → layout → activity_main.xml
- Click on Text view and type the XML layout code for designing the user interface

- Select a suitable available device or emulator to display the output
- Build and run the application to see the output

PROGRAM CODE

MainActivity.java

```
package com.example.tokendispenser;
import android.os.Bundle;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {
    EditText nameInput;
    Button getTokenButton;
    TextView tokenDisplay;
    ListView tokenListView;
    int tokenCount = 0;
    ArrayList<String> tokenList;
    ArrayAdapter<String> adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        nameInput = findViewById(R.id.nameInput);
        getTokenButton = findViewById(R.id.getTokenButton);
        tokenDisplay = findViewById(R.id.tokenDisplay);
        tokenListView = findViewById(R.id.tokenListView);
        tokenList = new ArrayList<>();
        adapter = new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, tokenList);
        tokenListView.setAdapter(adapter);
        getTokenButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String name = nameInput.getText().toString().trim();
                if (!name.isEmpty()) {
                    tokenCount++;
                    String tokenInfo = "Token " + tokenCount + " - " + name;
                    tokenDisplay.setText("Issued: " + tokenInfo);
                    tokenList.add(tokenInfo);
                    adapter.notifyDataSetChanged();
```

```
                    nameInput.setText("");
                } else {
                    Toast.makeText(MainActivity.this, "Please enter a name",
Toast.LENGTH_SHORT).show();
                }
            }
        });
    }}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">
    <TextView
        android:id="@+id/titleText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Token Dispenser"
        android:textSize="24sp"
        android:textStyle="bold" />
    <EditText
        android:id="@+id/nameInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name" />
    <Button
        android:id="@+id/getTokenButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Get Token" />
    <TextView
        android:id="@+id/tokenDisplay"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Issued: " />
    <ListView
        android:id="@+id/tokenListView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
```

```
</LinearLayout>
```

SCREENSHOTS

Figure 1: Initial State - Token Dispenser Interface

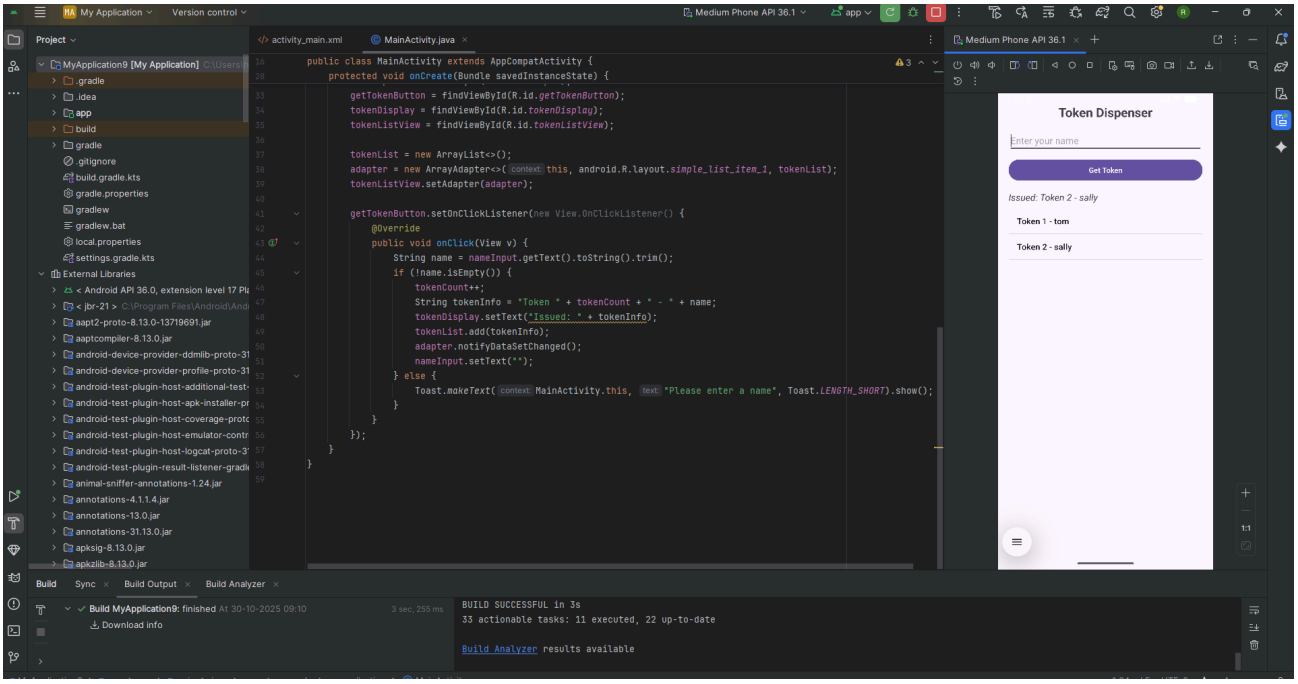
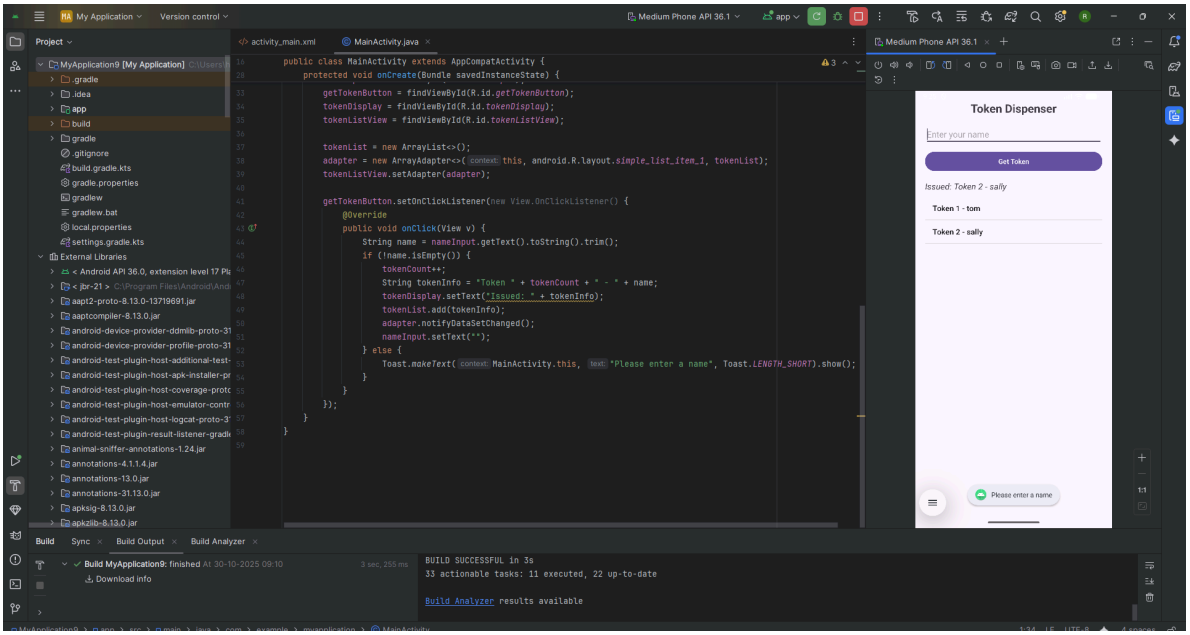


Figure 2: Name Entry Prompt - User Input Screen



TEST CASES

Test Case	Input Name	Expected Output	Expected ListView Update	Status
TC01	Sally	Token 1 - Sally	Token 1 - Sally	Pass
TC02	Tom	Token 2 - Tom	Token 1 - Sally, Token 2 - Tom	Pass
TC03	(Empty)	Toast Error	No update	Pass
TC04	John	Token 3 - John	Previous 2 + Token 3 - John	Pass

INFERENCE

Through the successful implementation of the Token Dispenser Android application, several key inferences can be drawn regarding Android application development principles and practices:

- **ListView and ArrayAdapter Management:** The implementation demonstrates effective use of ArrayAdapter for dynamic list management. The adapter automatically updates the ListView when data changes through the notifyDataSetChanged() method.
- **Event-Driven Programming:** The application effectively demonstrates Android's event-driven architecture through OnClickListener interfaces for button interactions.
- **Input Validation:** The isEmpty() check prevents empty tokens from being created, demonstrating defensive programming practices.
- **State Management:** The tokenCount variable maintains state across multiple user interactions, properly tracking token sequence.
- **XML-Based Layout Design:** The LinearLayout provides a clean, organized user interface with proper spacing and widget arrangement.
- **Separation of Concerns:** The clear separation between presentation layer (XML) and business logic (Java) facilitates maintainable application architecture.
- **Collection Framework Usage:** The ArrayList provides dynamic token storage with efficient insertion operations for queue management.

RESULT

The Android application for token dispensing has been successfully developed, implemented, and tested using Android Studio. The application fulfills all specified requirements by providing functionality to:

- Accept user names as input through EditText fields
- Generate unique sequential token numbers upon button click
- Display the currently issued token in a TextView component
- Maintain a dynamic list of all issued tokens in a ListView
- Validate user input and display error messages using Toast notifications

All test cases have been executed successfully, confirming the application's reliability and functionality. The application demonstrates fundamental Android development concepts including UI design using XML layouts, event handling with OnClickListener interfaces, dynamic list management using ArrayAdapter, and input validation mechanisms.

The experiment has effectively demonstrated the development process of a functional Android application for queue management and achieved the stated objectives of the laboratory exercise.